UNIVERSITY OF BRISTOL

Winter 2025 Examination Period

SCHOOL OF COMPUTER SCIENCE

Second Year Examination for the Degrees
of
Bachelor of Science
Master of Engineering
Master of Science

COMS20007W
Programming Languages and Computation

TIME ALLOWED:
2 Hours

Answers to COMS20007W: Programming Languages and Computation

Intended Learning Outcomes:

A grammar for association lists consists of:

- 6 terminal symbols:

$$\{ \quad \} \quad \text{literal} \quad \text{ident} \quad = \quad ;$$

- 4 nonterminal symbols: $Assoc$, $Value$, $Maplet$ and $Maplets$.
- 7 production rules:

$$\begin{array}{lrcl} (1) & Assoc & ::= & \{\ Maplets\ \} \\ (2) & Value & ::= & \text{literal} \\ (3) & & | & Assoc \\ (4) & Maplet & ::= & \text{ident} = Value \\ (5) & Maplets & ::= & Maplet \\ (6) & & | & Maplets\ ;\ Maplet \\ (7) & & | & \epsilon \end{array}$$

- The start symbol is $Assoc$.

Figure 1: A grammar for association lists.

Q1. This question is about syntax.

*(a) Consider the grammar in Figure 1. For each of the following, either give a derivation sequence to show that the string is in the language of the grammar, or state that the string is not derivable.

i. { }
ii. literal
iii. { foo = literal }
iv. { ident = { } }
v. { ; ident = literal }

[8 marks]

Solution:

i. $Assoc \rightarrow \{\ Maplets\ \} \rightarrow \{\ \}$

ii. not derivable

iii. not derivable

iv. $Assoc \rightarrow \{\ Maplets\ \} \rightarrow \{\ Maplet\ \} \rightarrow \{\ \text{ident} = Value\ \} \rightarrow \{\ \text{ident} = Assoc\ \} \rightarrow \{\ \text{ident} = \{\ \}\ \}$

v. $Assoc \rightarrow \{\ Maplets\ \} \rightarrow \{\ Maplets\ ;\ Maplet\ \} \rightarrow \{\ Maplets\ ;\ \text{ident} = Value\ \} \rightarrow \{\ Maplets\ ;\ \text{ident} = \text{literal}\ \} \rightarrow \{\ ;\ \text{ident} = \text{literal}\ \}$

** (b) Consider again the grammar in Figure 1. The nullability and first and follow sets for the nonterminals are as given below:

Nonterminal	Nullable	First	Follow
$Assoc$	×	{	} ;
$Value$	×	literal {	} ;
$Maplet$	×	ident	} ;
$Maplets$	✓	ident ;	} ;

i. For each of the right-hand-sides α of the 7 production rules in the grammar, compute First(α).

ii. Construct the parsing table for the grammar.

[12 marks]

Solution:

i. As a table:

Num	α	First(α)
1	$\{\ Maplets\ \}$	{
2	literal	literal
3	$Assoc$	{
4	ident $= Value$	ident
5	$Maplet$	ident
6	$Maplets\ ;\ Maplet$	ident, ;
7	ϵ	

ii. Parsing table:

NT	{	}	literal	ident	=	;
$Assoc$	1					
$Value$	3		2			
$Maplet$				4		
$Maplets$		7		5, 6		6, 7

** (c) Consider again the grammar of Figure 1. Give an equivalent grammar which is LL(1). You need not justify your answer.

[6 marks]

(cont.)

Solution:

$$
\begin{array}{rcl}
\mathit{Assoc} & ::= & \{\ \mathit{Maplets}\ \} \\
\mathit{Value} & ::= & \mathsf{literal} \\
 & | & \mathit{Assoc} \\
\mathit{Maplet} & ::= & \mathsf{ident} = \mathit{Value} \\
\mathit{Maplets} & ::= & B\ [;\ \mathit{Maplet}]^* \\
B & ::= & \mathit{Maplet} \\
 & | & \epsilon
\end{array}
$$

***(d) Consider the following grammar for ground Boolean formulas over the 7 terminal symbols true, false, $\wedge$, $\vee$, $\neg$,) and (:

$$B ::= \mathsf{true} \mid \mathsf{false} \mid B \wedge B \mid B \vee B \mid \neg B \mid (\ B\ )$$

Construct a context-free grammar to describe the subset of all ground Boolean formulas that evaluate to true. Boolean formulas are evaluated according to the usual meaning of the symbols $\wedge$ (and, conjunction), $\vee$ (or, disjunction) and $\neg$ (not, negation), with the understanding that:

- $\neg$ binds tighter than $\wedge$ which binds tighter than $\vee$
- $\wedge$ and $\vee$ associate to the right

Justify your construction by explaining informally what kinds of strings each of your nonterminals derive.

[8 marks]

Solution: It is important to stratify the grammar to avoid generating strings like $\neg$true $\wedge$ false whilst allowing $\neg$false $\wedge$ true.

$$
\begin{array}{rcl}
TD & ::= & TC \mid AC \vee TD \mid TC \vee AD \\
TC & ::= & TN \mid TN \wedge TC \\
TN & ::= & TL \mid \neg\ FN \\
TL & ::= & \mathsf{true} \mid (\ TD\ ) \\
FD & ::= & FC \mid FC \vee FD \\
FC & ::= & FN \mid FN \wedge AC \mid AN \wedge FC \\
FN & ::= & FL \mid \neg\ TN \\
FL & ::= & \mathsf{false} \mid (\ FD\ ) \\
AD & ::= & AC \mid AC \vee AD \\
AC & ::= & AN \mid AN \wedge AC \\
AN & ::= & AL \mid \neg\ AN \\
AL & ::= & \mathsf{true} \mid \mathsf{false} \mid (\ AD\ )
\end{array}
$$

Here, each nonterminal is coded XY, where:

- $X = T$ derives formulas that evaluate to true
- $X = F$ derives formulas that evaluate to false
- $X = A$ derives all formulas
- $Y = D$ derives disjunctive formulas (including the trivial disjunction of one disjunct)
- $Y = C$ derives conjunctive formulas (including the trivial conjunction of one conjunct)
- $Y = N$ derives negation formulas or atomic formulas
- $Y = L$ derives atomic formulas (literals or parenthesized formulas)

Q2. This question is about semantics.

*(a) Consider the following code fragments from the While language:

i. while $!(y \leq x)$ do $x \leftarrow x + x$

ii. $x = 3 + z$

iii. if $x \leq y$ then $x \leftarrow (y - x)$ else $x \leftarrow (x - y)$

For each of these fragments:

- Indicate whether it is an arithmetic expression, Boolean expression, or statement;
- If it is an expression, calculate its value with the store $[x \mapsto 2, y \mapsto 3]$. Your answer should make explicit reference to the denotation function.
- If it is a statement, determine the terminal store when the statement is executed with the initial store $[x \mapsto 1, y \mapsto 3]$ and give a trace leading to that store.

Solution:

i. This is a statement. The trace is:

$$\begin{aligned}
&\langle \text{while } !(y \leq x) \text{ do } x \leftarrow x + x + 1,\ [x \mapsto 1, y \mapsto 3]\rangle \\
&\quad \rightarrow \langle x \leftarrow x + x;\ \text{while } !(y \leq x) \text{ do } x \leftarrow x + x,\ [x \mapsto 1, y \mapsto 3]\rangle \\
&\quad \rightarrow \langle \text{while } !(y \leq x) \text{ do } x \leftarrow x + x,\ [x \mapsto 2, y \mapsto 3]\rangle \\
&\quad \rightarrow \langle x \leftarrow x + x;\ \text{while } !(y \leq x) \text{ do } x \leftarrow x + x,\ [x \mapsto 2, y \mapsto 3]\rangle \\
&\quad \rightarrow \langle \text{while } !(y \leq x) \text{ do } x \leftarrow x + x,\ [x \mapsto 4, y \mapsto 3]\rangle \\
&\quad \rightarrow [x \mapsto 4, y \mapsto 3]
\end{aligned}$$

ii. This is a Boolean expression.

$$\begin{aligned}
&[\![x = 3 + z]\!]_{\mathcal{B}}(\sigma) \\
&\quad = ([\![x]\!]_{\mathcal{A}}(\sigma) = [\![3 + z]\!]_{\mathcal{A}}(\sigma)) \\
&\quad = (\sigma(x) = 3 + \sigma(z)) \\
&\quad = (2 = 3 + 0) \\
&\quad = \bot
\end{aligned}$$

iii. This is a statement. The trace is:

$$\begin{aligned}
&\langle \text{if } x \leq y \text{ then } x \leftarrow (y - x) \text{ else } x \leftarrow (x - y),\ [x \mapsto 1, y \mapsto 3]\rangle \\
&\quad \rightarrow \langle x \leftarrow (y - x),\ [x \mapsto 1, y \mapsto 3]\rangle \\
&\quad \rightarrow [x \mapsto 2, y \mapsto 3]
\end{aligned}$$

[6 marks]

**(b) In this question, the grammar of arithmetic expressions have been extended with a new

language construct:

$$A ::= n \mid x \mid A + A \mid A - A \mid A * A \mid \mathsf{ToInt}(B)$$

where B refers to Boolean expressions.

The denotational semantics has been extended accordingly:

$$[\![\mathsf{ToInt}(e)]\!]_{\mathcal{A}}(\sigma) = \begin{cases} 1 & \text{if } [\![e]\!]_{\mathcal{B}}(\sigma) = \top \\ 0 & \text{otherwise} \end{cases}$$

i. Determine whether $\mathsf{ToInt}(e_1 \parallel e_2)$ is semantically equivalent to $\mathsf{ToInt}(e_1) + \mathsf{ToInt}(e_2)$ or not for all Boolean expressions $e_1, e_2 \in \mathcal{B}$. If they are semantically equivalent, prove so. If they are not, provide a counter-example.

ii. Determine whether $\mathsf{ToInt}(e) + \mathsf{ToInt}(!e)$ is semantically equivalent to 1 or not for all Boolean expression $e \in \mathcal{B}$. If they are semantically equivalent, prove so. If they are not, provide a counter-example.

Solution:

i. They are not semantically equivalent. Any two Boolean expressions that can both evaluate to $\top$ under the same state provide a counter-example.

ii. They are semantically equivalent.

Proof: Let $e \in \mathcal{B}$ be a Boolean expression and let $\sigma \in \mathsf{State}$ be a state.

- Let us suppose that $[\![e]\!]_{\mathcal{B}}(\sigma) = \top$. Therefore, $[\![\mathsf{ToInt}(e)]\!]_{\mathcal{A}}(\sigma) = 1$ and $[\![\mathsf{ToInt}(!e)]\!]_{\mathcal{A}}(\sigma) = 0$ by definition. Moreover,

$$\begin{aligned} &[\![\mathsf{ToInt}(e) + \mathsf{ToInt}(!e)]\!]_{\mathcal{A}}(\sigma) \\ &\quad = [\![\mathsf{ToInt}(e)]\!]_{\mathcal{A}}(\sigma) + [\![\mathsf{ToInt}(!e)]\!]_{\mathcal{A}}(\sigma) \\ &\quad = 1 + 0 \\ &\quad = 1 \end{aligned}$$

- Conversely, if $[\![b]\!]_{\mathcal{B}}(\sigma) = \bot$, then we have that $[\![\mathsf{ToInt}(e)]\!]_{\mathcal{A}}(\sigma) = 0$ and $[\![\mathsf{ToInt}(!e)]\!]_{\mathcal{A}}(\sigma) = 1$ by definition. In which case, $[\![\mathsf{ToInt}(e) + \mathsf{ToInt}(!e)]\!]_{\mathcal{A}}(\sigma) = 1$ analogously to the previous case.

In either case, $[\![\mathsf{ToInt}(e) + \mathsf{ToInt}(!e)]\!]_{\mathcal{A}}(\sigma) = 1$ and thus $\mathsf{ToInt}(e) + \mathsf{ToInt}(!e)$ is semantically equivalent to 1 as required.

[12 marks]

(cont.)

***(c) Consider the While program in Figure 2 and let us write S to refer to this program.

```
q ← 0;
r ← x;
while y ≤ r do
  r ← r − y;
  q ← q + 1
```

Figure 2: While Program for Euclidean Division.

i. Find a state $\sigma \in$ State such that the configuration $\langle S, \sigma \rangle$ is non-terminating.

ii. Find the strongest post-condition $P \subseteq$ State such that:

$$\{y \leq 0 \;\&\&\; y \leq r\} \text{ while } y \leq r \text{ do } \{r \leftarrow r - y;\; q \leftarrow q + 1\}\; \{P\}$$

Justify your answer by giving a loop invariant.

iii. Give a pre-condition $P \subseteq$ State such that which the configuration $\langle S, \sigma \rangle$ is non-terminating for any state σ that satisfies the pre-condition P. Justify your answer in reference to the post-condition calculated in part ii and the definition of a Hoare triple.

Solution:

i. Any state with $y \leq 0$ and $y \leq x$.

ii. The post-condition must be invalid, either $\bot$, or $!(y \leq r) \;\&\&\; y \leq r$, or some variation there of. The loop invariant is $y \leq 0$ and $y \leq r$.

iii. The pre-condition is $y \leq 0$ and $y \leq x$. Any stronger pre-condition is valid, but marks will be lost in the justification.

Justification: The solution to the previous part tells us that, if $\sigma \in$ State is some state that satisfies $y \leq 0 \;\&\&\; y \leq r$ and we have that

$$\langle \text{while } y \leq r \text{ do } \{r \leftarrow r - y;\; q \leftarrow q + 1\}, \sigma \rangle \rightarrow^* \sigma'$$

then σ' will satisfy $\bot$. However, no state can satisfy $\bot$, and thus we cannot have $\langle \text{while } y \leq r \text{ do } \{r \leftarrow r - y;\; q \leftarrow q + 1\}, \sigma \rangle \rightarrow^* \sigma'$ for any state σ'. In other words, this loop does not terminate when $y \leq 0$ and $y \leq r$. The pre-condition $y \leq 0$ and $y \leq x$ ensures that when the loop is reached then the state will satisfy $y \leq 0$ and $y \leq r$, and thus the program as a whole does not terminate under this condition.

[15 marks]

Q3. This question is about computability.

*(a) Show that the function $f : \mathbb{N} \rightharpoonup \mathbb{N}$ defined by

$$f(x)\begin{cases} \simeq x+1 & \text{if } x^{2^x} = 2025 \\ \simeq 1 & \text{if } x^{2^x} < 2025 \\ \uparrow & \text{otherwise} \end{cases}$$

is computable. *[7 marks]*

Solution: The function is computed by the following code with respect to x.

```
i := 0; j := 1; // invariant: j = 2^i and i >= 0
if (i <= x) { j := j * 2; i := i + 1 };

// at this point j = 2^x

i := 0; k := 1; // invariant k = x^i and i >= 0
if (i <= j) { k := k * x; i := i + 1};

// at this point j = 2^x and k = x^j = x^(2^x)

if (k = 2025) { x := x + 1 }
else if (k < 2025) { x := 1 }
else { while (true) do skip };
i := 0; j := 0; k := 0
```

Award 1 mark for correctly stating the input/output variable; 1 mark for setting all auxiliary variables to zero at the end; 4 marks for a mostly correct program; and 1 mark for the infinite loop when the output is undefined.

*(b) State whether each of the following statements is true or false.

(i) The set $\{10^{2^{10^2}}\}$ is undecidable.

(ii) The set of (encodings of) syntactically correct Java programs is decidable.

(iii) The empty set is undecidable.

(iv) Rice's Theorem is a statement about semi-decidable predicates.

(v) If a function is a surjection it must be computable.

[10 marks]

(cont.)

Solution: (i) False (ii) True (iii) False (iv) False (v) False. Award 2 marks for each correct answer.

** (c) Consider the predicate

$$U = \{\ulcorner S \urcorner \mid [\![S]\!]_{\mathtt{x}}(k) \simeq 1 \text{ for all } 0 \leq k \leq 10^5\}$$

Is U decidable? Why, or why not? *[8 marks]*

Solution: By Rice's Theorem. Award 1 mark for stating the name of the theorem, 4 marks for demonstrating that it applies (there is an S that is in the predicate and one that is not), and a further 2 marks for a fully correct argument. Alternative solutions by reduction are also acceptable.

***(d) Show that the predicate

$$V = \{\langle \ulcorner S \urcorner, \ulcorner T \urcorner \rangle \mid \text{ there exists } k \text{ such that } [\![S]\!]_{\mathtt{x}}(k) - [\![T]\!]_{\mathtt{x}}(k) \simeq 2\}$$

is undecidable. *[8 marks]*

Solution: By reduction from HALT. Define a code transformation $F : \textbf{Stmt} \times \mathbb{N} \to \textbf{Stmt} \times \textbf{Stmt}$ by

$$F(S, n) = (\texttt{x := 3}, (\texttt{x := n; S; x := 1}))$$

Clearly the reflection of F is computable, and

$$\langle \ulcorner S \urcorner, n \rangle \in \mathsf{HALT} \iff F(S, n) \in V$$

As the former is not decidable, the latter is not decidable either.

Award 2 marks for recognising that a reduction is the most appropriate proof method; 2 marks for constructing the reduction and arguing that it is computable; 2 marks for a correct argument that it is a reduction; and 1 mark for correctly stating the reduction property in this particular instance.

Reminder of Important Definitions

Grammars

A *Context Free Grammar (CFG)* consists of four components:

- An alphabet of *terminal* symbols.
- A finite, non-empty set of *non-terminal* symbols, disjoint from the terminals.
- A finite set of *production rules*.
- A designated non-terminal called the *start symbol*.

A *sentential form*, usually α, β, γ and so on, is just a finite sequence of terminals and nonterminals.

The sentential form α can make a *derivation step* to β, written $\alpha \to \beta$, just if:

- α has shape $\gamma_1 X \gamma_2$ and β has shape $\gamma_1 \delta \gamma_2$
- and there is a production rule $X ::= \delta$ in the grammar

A *derivation sequence* is a non-empty sequence of sentential forms α_1, α_2, ... α_{k-1}, α_k in which consecutive elements of the sequence are derivation steps:

$$\alpha_1 \to \alpha_2 \to \cdots \to \alpha_{k-1} \to \alpha_k$$

A sentential form β is *derivable* from α, written $\alpha \to^* \beta$ just if there is a derivation sequence starting with α and ending with β.

We say that a word w is in the *language of a grammar* G with start symbol S, and write $w \in L(G)$ just if $S \to^* w$.

Nullable

On nonterminals:

$$\text{Nullable}(X) \text{ iff } X \to^* \epsilon$$

On sentential forms:

$$\text{Nullable}_s(\alpha) = \begin{cases} \text{true} & \text{if } \alpha = \epsilon \\ \text{false} & \text{if } \alpha \text{ is of shape } a\beta \\ \text{Nullable}(X) \wedge \text{Nullable}_s(\beta) & \text{if } \alpha \text{ is of shape } X\beta \end{cases}$$

(cont.)

First

On nonterminals:

$$\mathsf{First}(X) = \{a \mid \exists \beta.\, X \to^* a\beta\}$$

On sentential forms:

$$\mathsf{First}_s(\alpha) = \begin{cases} \emptyset & \text{if } \alpha = \epsilon \\ \{a\} & \text{if } \alpha \text{ is of shape } a\beta \\ \mathsf{First}(X) & \text{if } \alpha \text{ is of shape } X\beta \text{ and } \neg\mathsf{Nullable}(X) \\ \mathsf{First}(X) \cup \mathsf{First}_s(\beta) & \text{if } \alpha \text{ is of shape } X\beta \text{ and } \mathsf{Nullable}(X) \end{cases}$$

Follow

On nonterminals:

$$\mathsf{Follow}(X) = \{a \mid \exists \alpha\beta.\, S \to^* \alpha X a \beta\}$$

Parse Tables and LL(1)

We define the *parsing table*, usually T, for a given grammar as a 2d array indexed by pairs of a nonterminal and a terminal. Each entry $T[X, a]$ is a set of production rules from the grammar, such that some rule $X \longrightarrow \beta$ is in the set $T[X, a]$ just if, either:

1. $a \in \mathsf{First}_s(\beta)$
2. or, $\mathsf{Nullable}_s(\beta)$ and $a \in \mathsf{Follow}(X)$

A grammar whose parsing table contains at most one rule in each cell is called *LL(1)*.

Abstract Syntax of Arithmetic Expressions

An *arithmetic expression* is a tree described by the following grammar:

$$A ::= n \mid x \mid A + A \mid A - A \mid A * A$$

where n ranges over integer literals, and x ranges over variables. Parentheses are used to resolve ambiguity and to indicate the structure of the tree. We write $\mathcal{A}$ for the set of arithmetic expressions.

Abstract Syntax of Boolean Expressions

A *Boolean expression* is a tree described by the following grammar.

$$B ::= \mathsf{false} \mid \mathsf{true} \mid !B \mid B \;\&\&\; B \mid B \parallel B \mid A = A \mid A \leq A$$

Parentheses are used to resolve ambiguity and to indicate the structure of the tree. We write $\mathcal{B}$ for the set of Boolean expressions.

Abstract Syntax of Statements

A *statement* is a tree described by the following grammar:

$$S ::= \text{skip} \mid x \leftarrow A \mid S; S \mid \text{if } B \text{ then } S \text{ else } S \mid \text{while } B \text{ do } S$$

Braces "$\{\cdots\}$" are used to resolve ambiguity and to indicate the structure of the tree. We write $\mathcal{S}$ for the set of statements.

States

A *state* is a total function from the set State $=$ Var $\to \mathbb{Z}$, where Var is the set of variables. We write $[x_1 \mapsto v_1, x_2 \mapsto v_2, \ldots, x_n \mapsto v_n]$ to indicate the state that maps the variable $x_i \in$ Var to the value $v_i \in \mathbb{Z}$ for all $i \leq n$. By convention, any variable not explicitly mentioned by a given state σ is assigned the value 0.

For a given state $\sigma \in$ State, we write $\sigma[x \mapsto v]$ for some variable $x \in$ Var and $v \in \mathbb{Z}$ to denote the state that maps the variable x to v and any other variable y to the value $\sigma(y)$.

Semantics of Arithmetic Expressions

The denotation function for arithmetic expressions $[\![\cdot]\!]_{\mathcal{A}} \in \mathcal{A} \to (\text{State} \to \mathbb{Z})$, which is defined by recursion in Figure 3. We say that two arithmetic expressions $e_1, e_2 \in \mathcal{A}$ are *semantically equivalent* if, and only if, $[\![e_1]\!]_{\mathcal{A}}(\sigma) = [\![e_2]\!]_{\mathcal{A}}(\sigma)$ for all states $\sigma \in$ State.

$$\begin{aligned}
[\![n]\!]_{\mathcal{A}}(\sigma) &= n \\
[\![x]\!]_{\mathcal{A}}(\sigma) &= \sigma(x) \\
[\![e_1 + e_2]\!]_{\mathcal{A}}(\sigma) &= [\![e_1]\!]_{\mathcal{A}}(\sigma) + [\![e_2]\!]_{\mathcal{A}}(\sigma) \\
[\![e_1 - e_2]\!]_{\mathcal{A}}(\sigma) &= [\![e_1]\!]_{\mathcal{A}}(\sigma) - [\![e_2]\!]_{\mathcal{A}}(\sigma) \\
[\![e_1 * e_2]\!]_{\mathcal{A}}(\sigma) &= [\![e_1]\!]_{\mathcal{A}}(\sigma) \cdot [\![e_2]\!]_{\mathcal{A}}(\sigma)
\end{aligned}$$

Figure 3: Definition of the denotational semantics of arithmetic expressions.

Semantics of Boolean Expressions

The denotation function for Boolean expressions $[\![\cdot]\!]_{\mathcal{B}} \in \mathcal{B} \to (\text{State} \to \mathbb{B})$ is defined by recursion in Figure 4. We say that two Boolean expressions $e_1, e_2 \in \mathcal{B}$ are *semantically equivalent* if, and only if, $[\![e_1]\!]_{\mathcal{B}}(\sigma) = [\![e_2]\!]_{\mathcal{B}}(\sigma)$ for all states $\sigma \in$ State.

(cont.)

$$
\begin{aligned}
[\![\mathsf{false}]\!]_{\mathcal{B}}(\sigma) &= \bot \\
[\![\mathsf{true}]\!]_{\mathcal{B}}(\sigma) &= \top \\
[\![!e]\!]_{\mathcal{B}}(\sigma) &= \neg[\![e]\!]_{\mathcal{B}}(\sigma) \\
[\![e_1 \mathbin{\&\&} e_2]\!]_{\mathcal{B}}(\sigma) &= [\![e_1]\!]_{\mathcal{B}}(\sigma) \wedge [\![e_2]\!]_{\mathcal{B}}(\sigma) \\
[\![e_1 \parallel e_2]\!]_{\mathcal{B}}(\sigma) &= [\![e_1]\!]_{\mathcal{B}}(\sigma) \vee [\![e_2]\!]_{\mathcal{B}}(\sigma) \\
[\![e_1 = e_2]\!]_{\mathcal{B}}(\sigma) &= [\![e_1]\!]_{\mathcal{A}}(\sigma) = [\![e_2]\!]_{\mathcal{A}}(\sigma) \\
[\![e_1 \leq e_2]\!]_{\mathcal{B}}(\sigma) &= [\![e_1]\!]_{\mathcal{A}}(\sigma) \leq [\![e_2]\!]_{\mathcal{A}}(\sigma)
\end{aligned}
$$

Figure 4: Definition of the denotational semantics of Boolean expressions.

Semantics of Statements

The small-step operational semantics relation $\rightarrow \subseteq \mathcal{C} \times \mathcal{C}$ is defined by the rules in Figure 5 where the set of configurations $\mathcal{C}$ is $(\mathcal{S} \times \mathsf{State}) \cup \mathsf{State}$.

$$\frac{}{\langle \mathsf{skip},\, \sigma\rangle \rightarrow \sigma} \qquad \frac{}{\langle x \leftarrow e,\, \sigma\rangle \rightarrow \sigma[x \mapsto [\![e]\!]_{\mathcal{A}}(\sigma)]}$$

$$\frac{\langle S_1,\, \sigma_1\rangle \rightarrow \langle S_1',\, \sigma_2\rangle}{\langle S_1; S_2,\, \sigma_1\rangle \rightarrow \langle S_1';\ S_2,\, \sigma_2\rangle} \qquad \frac{\langle S_1,\, \sigma_1\rangle \rightarrow \sigma_2}{\langle S_1; S_2,\, \sigma_1\rangle \rightarrow \langle S_2,\, \sigma_2\rangle}$$

$$\frac{}{\langle \mathsf{if}\ e\ \mathsf{then}\ S_1\ \mathsf{else}\ S_2,\, \sigma\rangle \rightarrow \langle S_1,\, \sigma\rangle}\ [\![e]\!]_{\mathcal{B}}(\sigma) = \top$$

$$\frac{}{\langle \mathsf{if}\ e\ \mathsf{then}\ S_1\ \mathsf{else}\ S_2,\, \sigma\rangle \rightarrow \langle S_2,\, \sigma\rangle}\ [\![e]\!]_{\mathcal{B}}(\sigma) = \bot$$

$$\frac{}{\langle \mathsf{while}\ e\ \mathsf{do}\ S,\, \sigma\rangle \rightarrow \langle S;\ \mathsf{while}\ e\ \mathsf{do}\ S,\, \sigma\rangle}\ [\![e]\!]_{\mathcal{B}}(\sigma) = \top$$

$$\frac{}{\langle \mathsf{while}\ e\ \mathsf{do}\ S,\, \sigma\rangle \rightarrow \sigma}\ [\![e]\!]_{\mathcal{B}}(\sigma) = \bot$$

Figure 5: Definition of the operational semantics of statements.

Hoare Triples

A Hoare triple $\{P\}\ S\ \{Q\}$ for $P, Q \subseteq \mathsf{State}$ asserts that, for any state $\sigma \in \mathsf{State}$, if $\sigma \in P$ and $\langle S,\, \sigma\rangle \rightarrow^* \sigma'$, then $\sigma' \in Q$. The sets P and Q can be represented as Boolean expressions extended with quantifiers.

The rules for constructing Hoare triples are given in Figure 6.

$$\frac{}{\{P\}\ \mathsf{skip}\ \{P\}} \qquad \frac{}{\{P\}\ x \leftarrow e\ \{\exists x'.\, P[x'/x]\ \&\&\ x = e[x'/x]\}}$$

$$\frac{\{P\}\ S_1\ \{Q\} \quad \{Q\}\ S_2\ \{R\}}{\{P\}\ S_1; S_2\ \{R\}} \qquad \frac{\{P\ \&\&\ e\}\ S_1\ \{Q_1\} \quad \{P\ \&\&\ !e\}\ S_3\ \{Q_2\}}{\{P\}\ \mathsf{if}\ e\ \mathsf{then}\ S_1\ \mathsf{else}\ S_2\ \{Q_1 \parallel Q_2\}}$$

$$\frac{\{P\ \&\&\ e\}\ S\ \{P\}}{\{P\}\ \mathsf{while}\ e\ \mathsf{do}\ S\ \{P\ \&\&\ !e\}} \qquad \frac{\{P_1\}\ S\ \{Q_1\} \quad \begin{array}{l} P_2 \subseteq P_1 \\ Q_1 \subseteq Q_2 \end{array}}{\{P_2\}\ S\ \{Q_2\}}$$

Figure 6: Rules of Hoare logic.

Computable Functions

We write $[\mathtt{x} \mapsto n]$ for the state that maps the variable x to the number $n \in \mathbb{N}$, and every other variable to 0.

A 'while' program S *computes* a partial function $f : \mathbb{N} \rightharpoonup \mathbb{N}$ (with respect to x) just if $f(m) \simeq n$ exactly when $\langle S, [\mathtt{x} \mapsto m]\rangle \Downarrow [\mathtt{x} \mapsto n]$.

A function $f : \mathbb{N} \rightharpoonup \mathbb{N}$ is *computable* just if there is a program S that computes f with respect to the variable x.

Predicates

The *characteristic function* of U is the function

$$\chi_U : \mathbb{N} \to \mathbb{N}$$
$$\chi_U(n) = \begin{cases} 1 & \text{if } n \in U \\ 0 & \text{if } n \notin U \end{cases}$$

The *semi-characteristic function* of U is the partial function

$$\xi_U : \mathbb{N} \rightharpoonup \mathbb{N}$$
$$\xi_U(n) \begin{cases} \simeq 1 & \text{if } n \in U \\ \uparrow & \text{otherwise} \end{cases}$$

A predicate $U \subseteq \mathbb{N}$ is *decidable* just if its characteristic function $\chi_U : \mathbb{N} \to \mathbb{N}$ is computable.

The 'while' program that computes the characteristic function χ_U of a predicate $U \subseteq \mathbb{N}$ is called a *decision procedure*. Any predicate for which there is no decision procedure is called *undecidable*.

(cont.)

A predicate $U \subseteq \mathbb{N}$ is *semi-decidable* just if its semi-characteristic function ξ_U is computable.

The *Halting Problem* is the following predicate:

$$\mathsf{HALT} = \{\langle \ulcorner S \urcorner, n\rangle \mid [\![S]\!]_{\mathtt{x}}(n) \downarrow\}$$

Bijections

A function $f : A \to B$ is *injective* (or 1-1) just if for any $a_1, a_2 \in \mathcal{A}$ we have that $f(a_1) = f(a_2)$ implies $a_1 = a_2$. We sometimes write $f : A \rightarrowtail B$ whenever f is an injection.

A function $f : A \to B$ is *surjective* just if for any $b \in \mathcal{B}$ there exists $a \in \mathcal{A}$ such that $f(a) = b$. We sometimes write $f : A \twoheadrightarrow B$ whenever f is a surjection.

A function $f : A \to B$ is a *bijection* just if it is both injective and surjective.

Let $f : A \to B$ be a function. f is an *isomorphism* just if it has an *inverse*. That is, if there exists a function $f^{-1} : B \to A$ such that:

- for all $a \in \mathcal{A}$ we have $f^{-1}(f(a)) = a$
- for all $b \in \mathcal{B}$ we have $f(f^{-1}(b)) = b$

Encoding Data

A *pairing function* is a bijection $\mathbb{N} \times \mathbb{N} \xrightarrow{\cong} \mathbb{N}$. We assume that we have a fixed pairing function

$$\langle -, - \rangle : \mathbb{N} \times \mathbb{N} \xrightarrow{\cong} \mathbb{N}$$

with the following inverse:

$$\mathsf{split} : \mathbb{N} \xrightarrow{\cong} \mathbb{N} \times \mathbb{N}$$

Reflections

Suppose we have two bijections:

$$\phi : A \xrightarrow{\cong} \mathbb{N} \quad \psi : B \xrightarrow{\cong} \mathbb{N}$$

The *reflection* of $f : A \rightharpoonup B$ under (ϕ, ψ) is the function

$$\tilde{f} : \mathbb{N} \rightharpoonup \mathbb{N}$$
$$\tilde{f}(n) = \psi(f(\phi^{-1}(n)))$$

Gödel Numbering

Let **Stmt** be the set of Abstract Syntax Trees of While. We assume that we have a Gödel numbering

$$\ulcorner - \urcorner : \textbf{Stmt} \xrightarrow{\cong} \mathbb{N}$$

which encodes While programs as natural numbers.

A *code transformation* is a function $f : \textbf{Stmt} \to \textbf{Stmt}$.

Universal Function

The *universal function*, U, is defined as follows:

$$U : \mathbf{Stmt} \times \mathbb{N} \rightharpoonup \mathbb{N}$$
$$U(P, n) = [\![P]\!]_{\mathtt{x}}(n)$$

Reductions

Let $U, W \subseteq \mathbb{N}$ be predicates, and let $f : \mathbb{N} \to \mathbb{N}$. The function f is a *many-one reduction* from U to W just if it is computable, and it is also the case that

$$n \in U \Leftrightarrow f(n) \in W$$

We may write $f : U \lesssim V$ (read "f is a reduction from U to V").

END OF PAPER